

Byakugan

Complete Project Handbook

360° vision into any codebase: what it does, how it is organized, which concepts it uses, and what breaks if you change it, personalized to the reader.

Version	1.1 (adds report detail and the optional chat widget)
Prepared for	The Byakugan team
Event	IBM Bob 2.0 Hackathon
Duration	48 hours
Team size	2 developers

This handbook is a living document. Update it as decisions are made during the hackathon. When in doubt, this document is the source of truth. Diagrams and data schemas were added to the original text; schemas marked “proposed” are design suggestions.

Welcome, Reader

This document explains everything about Byakugan: what it is, why we are building it, what it does, how it works, and how we will build it in 48 hours. Terms are explained where they first appear. Nothing assumes prior knowledge of hackathons, Node.js, or IBM Bob.

Only 10 minutes? Read Part 1 (What we are building), Part 2 (The problem), and Part 3 (The solution). That gives the full picture; everything else is detail to read as needed.

What changed in v1.1

Area	Change
Chat widget	New stretch goal (section 4.6, FR-9, and the hour-28 checkpoint in 11.4). Optional: the core build does not depend on it.
The report	Part 7 rewritten: how the report works technically, design decisions, data flow, and sharing checks.
Priorities	Explicit P0 / Stretch / Pitch-only build table (3.3).
Security	The chat, if built, uses analysis data only (no raw source), a rate limit, and a server-side credential.
Plan	Phase 0 adds a 15-minute feasibility check; Day 2 adds a go/no-go checkpoint with a time-box.

Contents

1. What we are building
2. The problem
3. The solution
4. Features explained in detail
5. Functional requirements
6. Non-functional requirements
7. The report
8. The CLI
9. Architecture
10. Technology
11. The 48-hour plan
12. Team roles
13. Deliverables and success metrics
14. Glossary
15. One-page summary

Part 1. What We Are Building

1.1 The one-sentence pitch

Byakugan is a command-line tool that gives developers 360° vision into any codebase: explaining what it does, how it is organized, what concepts it uses, and what breaks if you change something, all personalized to the reader's skill level.

1.2 What that actually means

Imagine you join a new team and are handed a project with 400 files you have never seen. Today you would spend one to two weeks working out where things are: opening random files, asking senior engineers, reading stale docs. By the time you can make a real change, two weeks have passed.

With Byakugan you run one command:

```
byakugan analyze ./project
```

Two minutes later you open a report in your browser. It tells you what the project does, how the modules connect, which design patterns the code uses (explained at your level), and what breaks if you change a specific file. You understand the codebase in 30 minutes instead of two weeks.

1.3 What Byakugan is not

Question	Answer
Is it an IDE?	No. You keep using VS Code, IntelliJ, or whatever you already use.
Does it replace your editor?	No. It works alongside your editor.
Does it modify code?	No. Read-only by default. It produces reports.
Does it require internet?	Yes, for analysis. The report works offline afterward.
Does it support all languages?	No. We pick one language family (likely JavaScript/Node.js).
Is it a chatbot?	No. It is a structured tool with specific commands. An optional chat widget inside the report is a stretch goal (4.6).
Does it host your code?	No. Only analysis output is stored. Source code stays local.

1.4 The three parts of Byakugan

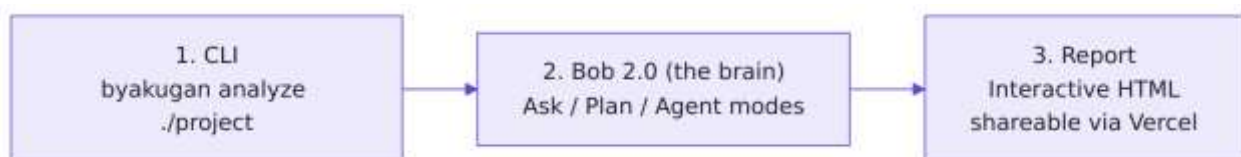


Figure 1. The three components of Byakugan

The flow, step by step:

1. You open a terminal and type `byakugan analyze ./my-project`.
 2. Byakugan sends the code to Bob 2.0.
 3. Bob reads the code and produces structured explanations.
 4. Byakugan turns those explanations into a web page.
 5. You open the page in your browser and read it, and can share it with a link.
- That is the entire tool: no IDE plugins, no complex setup. One command, one report.

Part 2. The Problem

2.1 The universal pain

Every developer eventually gets handed code they did not write: a new job, a legacy system nobody fully understands, a contractor joining mid-project, or an open-source contribution. The pain is always the same: you do not know where to start.

2.2 The four walls

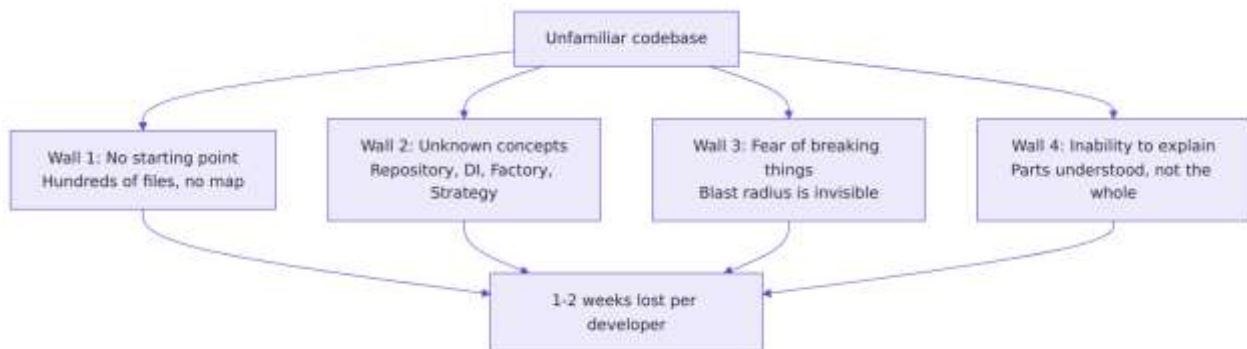


Figure 2. The four walls of an unfamiliar codebase

Wall 1: No starting point. Hundreds of files and no map. You open `index.js`, then the next file, with no idea which files matter, which are dead, or which hold the real logic.

Wall 2: Unknown concepts. The code uses Repository, Dependency Injection, Factory, Strategy. You may have heard the terms but cannot explain them, judge whether they are used well, or apply them.

Wall 3: Fear of breaking things. You want to fix a bug but do not know what depends on the function. Change one line and three features may break. There is no way to see the blast radius without tracing every call by hand.

Wall 4: Inability to explain. Even after reading the code you cannot articulate what it does. Asked “how does the refund flow work?”, you understand parts but not the whole.

2.3 Why existing tools fail

Tool	What it does	Why it is not enough
IDE search	Finds files and symbols	Does not explain why code exists or how it is designed
AI chat (ChatGPT, etc.)	Answers general questions	Generic answers without codebase context
Documentation	Describes features	Usually stale, incomplete, or missing
Code review tools	Review changes	Act after you have already written code
Static analysis tools	Find bugs, lint issues	Do not teach concepts or adapt to your level

None of these produce a personalized, layered understanding of a codebase that teaches concepts in context and warns you before you break something.

2.4 The cost of the problem

- 1–2 weeks of lost productivity per developer per new codebase.
- Delayed first contribution: they cannot help until they understand.
- Rework: code written without understanding gets rewritten.
- Senior engineers spend hours answering questions that could be self-served.
- Bugs introduced because the developer did not know what depended on what.

For a team of 5 developers this is roughly 10 person-weeks per new codebase, about two full months of one developer's time.

Part 3. The Solution

3.1 What Byakugan does

Point Byakugan at any codebase and it produces a personalized, interactive report that explains the code at your level. Six commands cover the whole interaction:

```
byakugan init           # 5 questions, profiles you
byakugan analyze ./project # analyzes the codebase
byakugan open           # opens the report in your browser
byakugan explain ./file.js # explains one file at your level
byakugan impact ./file.js # shows what breaks if you change this
byakugan share          # gets a shareable link
```

3.2 The five powers

Every feature maps to a power of the Byakugan eye from anime, and each feature is a section of the report.

Byakugan power	Our feature	What it does	Hackathon scope
X-ray vision	Skill Profiling	Sees who you are, adapts to you	Build
360° vision	Adaptive Explanations	Explains the same code differently per reader	Build
See the chakra network	Architecture Map	Visualizes how modules connect	If time allows
See through illusions	Technique Cards	Identifies design patterns in context	Pitch only
See great distances	Impact Sight	Tells you what breaks before you change it	Must build (centerpiece)

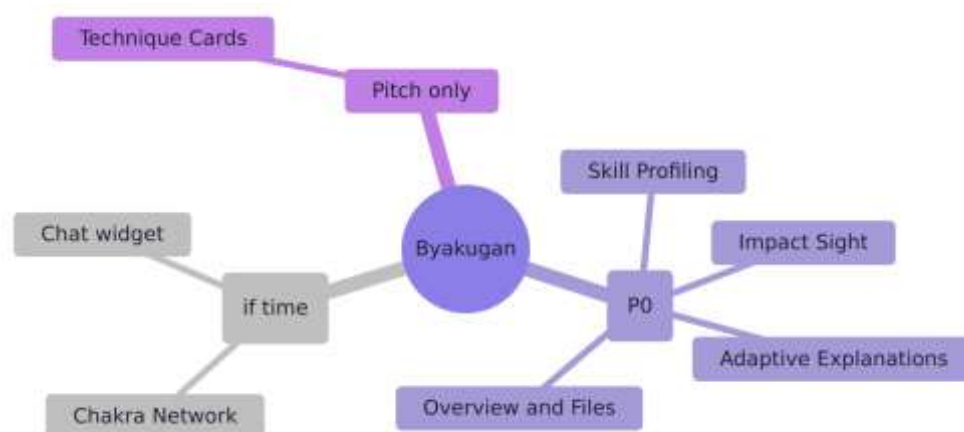


Figure 3. Feature scope for the hackathon

3.3 Build priorities

Not everything is equal. **P0** must be built. **Stretch** is attempted only if the core is stable at the hour-28 checkpoint. **Pitch only** appears as a mockup in the deck.

Item	Priority	What to build
Skill Profiling	P0	Five questions, saved profile
Overview	P0	Full, with the skill toggle
Adaptive Explanations	P0	Three pre-computed levels, instant toggle
Impact Sight	P0	Full; the demo centerpiece
Files	P0	Basic file browser with explanations
Chakra Network	Stretch	Static Mermaid diagram; cut if time is short
Chat widget	Stretch	Floating button, streaming answers (see 4.6)
Techniques	Pitch only	Mockup in the deck

Part 4. Features Explained in Detail

4.1 Feature 1: Skill Profiling

Before analyzing any code, Byakugan asks five questions about your background and saves the answers to `.byakugan/profile.json`. Every explanation afterward adapts to who you are. The same code needs different explanations for different people: without profiling, explanations are either too basic or too dense.

#	Question	Options
1	Overall experience	Beginner / Intermediate / Senior
2	Familiarity with the project's language	New / Comfortable / Expert
3	Familiarity with the project's domain	New / Some / Expert
4	Preferred learning style	Examples first / Theory first / Diagrams first
5	Your goal	Onboarding / Safe modification / Learn concepts

Proposed `profile.json` schema:

```
{
  "experience": "beginner | intermediate | senior",
  "language":   "new | comfortable | expert",
  "domain":     "new | some | expert",
  "style":      "examples | theory | diagrams",
  "goal":       "onboarding | safe-modification | learn-concepts",
  "updatedAt":  "2026-01-01T10:00:00Z"
}
```

Same code, three levels

Level	Explanation of a UserRepository
Beginner	“This is a Repository. Think of it like a librarian: you ask for a book, they find it. You do not need to know where the books are stored. In this code, UserRepository does the same thing with users.”
Intermediate	“This module uses the Repository pattern to abstract data access. UserRepository exposes <code>findById()</code> and <code>save()</code> but never leaks the database client, so you can swap the database without changing business logic.”
Senior	“Repository pattern, with a caveat: the interface is generic yet the implementation has a leaky abstraction. <code>findByEmail</code> is user-specific but lives in a generic contract. Worth refactoring if you extend this.”

4.2 Feature 2: Adaptive Explanations

For every file, function, or concept, Byakugan produces an explanation calibrated to your profile, either in the terminal (`byakugan explain ./src/services/UserService.js`) or by clicking any file in the report. Generic AI explanations ignore context; Byakugan does not. A beginner gets a friendly walkthrough, a senior gets tradeoffs and edge cases.

The killer interactive feature: in the report, a dropdown toggles the skill level and the same file re-explains itself instantly. All three levels are pre-computed by Bob during analysis, so toggling only swaps which version is displayed. No API calls.

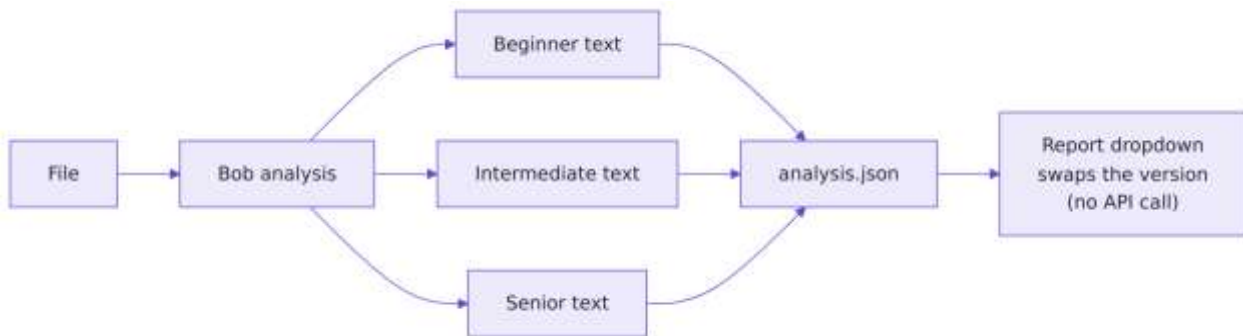


Figure 4. Pre-computed explanation levels power the instant toggle

4.3 Feature 3: Chakra Network (Architecture Map)

A visual diagram showing how modules connect: boxes are modules, lines show who calls whom. In the anime the Byakugan sees the hidden flow of chakra inside a person; our tool sees the hidden data flow inside a codebase. You cannot understand a project by reading files one at a time; you need to see its shape.

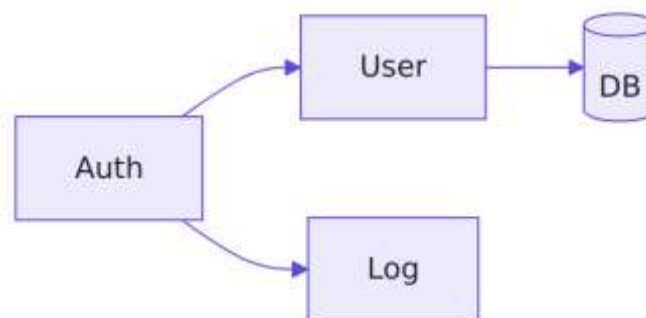


Figure 5. Example chakra network

Priority: stretch. We build a simple static version if time allows. The full interactive version is a roadmap item.

4.4 Feature 4: Techniques (Concept Cards)

For every design pattern, principle, or domain concept the code uses, Byakugan generates a technique card containing:

- **Name**, for example Repository Pattern.
- **What it is**, a plain-language definition.
- **Where it appears**, as file:line references in this codebase.
- **Why it matters**, the problem it solves.
- **Without it**, what would go wrong.
- **Tiny example**, a 10-line standalone snippet.

Field	Technique: Repository Pattern
What it is	Separates data access from business logic.
Where it appears	/src/repositories/UserRepo.js, /src/repositories/OrderRepo.js
Why it matters	You can swap the database without touching business logic.
Without it	Business logic is coupled to the database, making changes risky.

Concepts stick when tied to code you are already reading. **Priority: pitch only.** We show a mockup in the pitch deck but do not build it for the hackathon.

4.5 Feature 5: Impact Sight (Modification Guide)

For any file or function, Byakugan produces an impact report covering: what it does; direct dependents (with line numbers); indirect dependents; test coverage and gaps; blast radius; a suggested safe approach (Bob uses Plan Mode); and warnings about risky patterns. No other tool does this well: a developer points at a function and sees exactly what they are touching before touching it. **Priority: must build. This is the demo's centerpiece.**

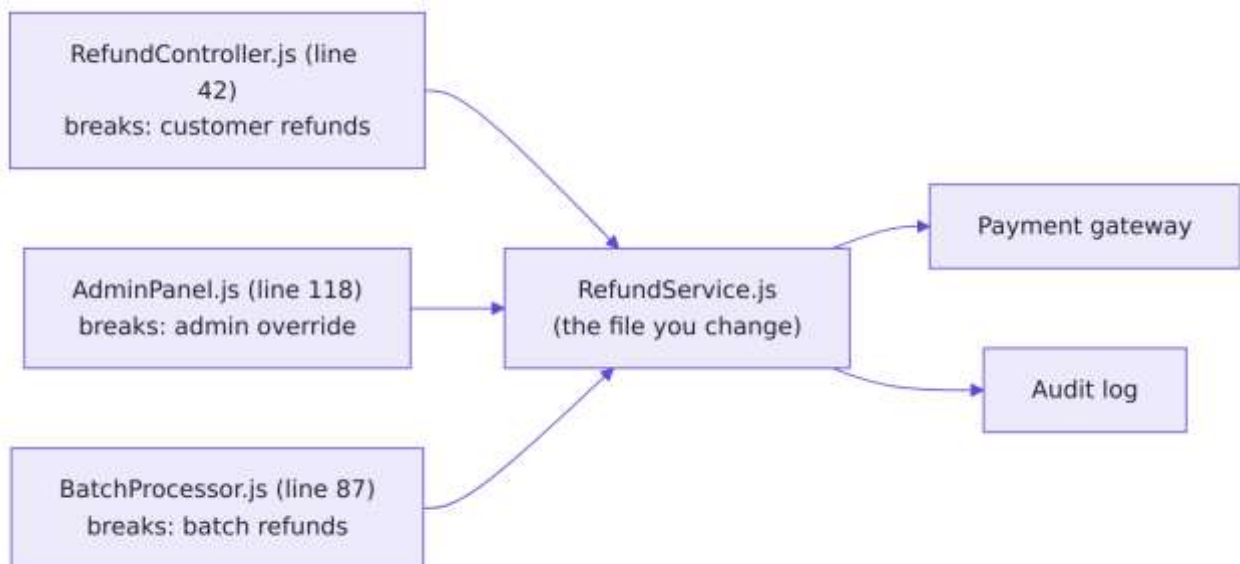


Figure 6. Impact Sight: direct dependents and what each one breaks

Example output:

```

IMPACT REPORT: RefundService.js

WHAT IT DOES
  Processes customer refunds, validates the order,
  calls the payment gateway, and logs the action.

WHO DEPENDS ON IT (direct)
  -> RefundController.js      (line 42)
  -> AdminPanel.js           (line 118)
  -> BatchProcessor.js       (line 87)

TEST COVERAGE
  [ok] Happy path tested
  [gap] Error handling NOT tested
  [gap] Invalid refund NOT tested

BLAST RADIUS IF BROKEN
  - Customer refunds stop working
  - Admin override stops working
  - Batch refunds stop working

SUGGESTED APPROACH
  1. Write tests for the 2 uncovered cases first
  2. Make the change
  3. Run the full test suite

WARNING
  This function mutates the order object in place. If you change
  it to return a new object, 3 callers will break silently.

```

4.6 Feature 6: Chat Widget (Stretch Goal)

Status: stretch goal. The chat is not part of the core build. We attempt it only if the core is finished and stable at the hour-28 checkpoint (see 11.4). The report is designed so the chat can be added, or left out, without changing anything else.

A floating button in the corner of every page opens a chat panel. You can ask “Why does this function call `validateOrder` twice?” and Bob answers in context, in read-only mode, at your skill level.

What the chat knows, and what it cannot do

The chat knows	The chat cannot
The analysis data: overview, summaries, dependencies, explanations	Modify code (Ask Mode is read-only)
Your skill profile	Run commands
The file you are currently reading	Access files outside the analyzed codebase, or see raw source (it works from the analysis)

How it works

The widget sends your message to a **Vercel serverless function** (`/api/chat.js`), which adds the relevant slice of analysis data and forwards the question to Bob. The answer is **streamed**

back token by token, which keeps the interface responsive and stays well inside function time limits.

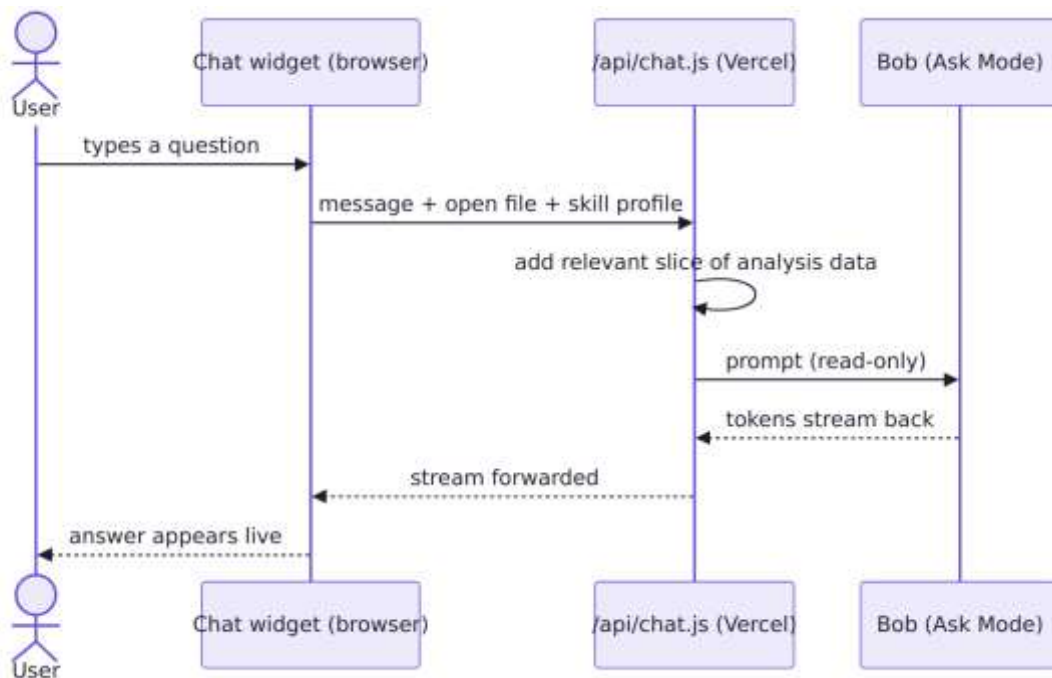


Figure 7. Chat widget request path (stretch)

Built to be optional

- **Isolated code:** the widget lives in `chat.js` and the server side in `api/chat.js`. No other file imports them.
- **Off by default:** the report loads the chat only when `chat.enabled` is true in the analysis metadata.
- **Placeholder in the template:** `index.html` includes an empty `chat-root` element from day one, so adding the widget later needs no layout rework.
- **No extra analysis:** the chat reads data the analysis already produces.
- **Fails quietly:** if the endpoint is missing or unreachable, the button is hidden and every other section works as normal.

Minimum viable version (definition of done)

- A floating button opens a chat panel on every page.
- A message reaches Bob and the answer streams back.
- Answers use the current file and the user's skill profile.
- Errors show a friendly message; the endpoint is rate-limited.

Anything beyond this (conversation history, source citations, suggested questions) is out of scope for the hackathon.

Risks and how we contain them

Risk	Mitigation
Bob access from a server: interactive SSO cannot run inside a serverless function.	Phase 0 feasibility check (15 minutes): confirm a non-interactive credential exists. If not, the chat is cut with no loss to the core.
Function time limits.	Stream the response. Vercel's current documentation lists a much higher default maximum for Hobby than the old 10-second figure, but limits differ for older projects, so verify in the dashboard during setup.
Anyone with the share link can call the endpoint and spend Bob quota.	Rate limit per visitor and require a simple token embedded in the page. Enable the chat for the demo only.
Source code leaving the machine.	The chat context is analysis data only, never raw source files.
Cost and latency of large context per message.	Send only the relevant slice (current file, its neighbors, the overview), not the whole <code>analysis.json</code> .
Chat needs internet, unlike the rest of the report.	Hide or disable the button when offline.

Part 5. Functional Requirements

Functional requirements describe what the tool must do. Every requirement is testable.

FR-1 Skill Profiling

ID	Requirement
FR-1.1	The CLI must ask 5 profiling questions on first run.
FR-1.2	Answers must be saved to <code>.byakugan/profile.json</code> .
FR-1.3	The profile must be reusable across analyses.
FR-1.4	The user must be able to update it via <code>byakugan profile update</code> .
FR-1.5	The profile must influence every subsequent explanation.

FR-2 Codebase Analysis

ID	Requirement
FR-2.1	<code>byakugan analyze <path></code> must accept any folder path.
FR-2.2	Analysis must complete in under 5 minutes for a 200-file repo.
FR-2.3	Analysis must produce structured JSON output.
FR-2.4	Analysis must use Bob 2.0 in Ask Mode (read-only).
FR-2.5	Analysis must spawn subagents for parallel exploration.
FR-2.6	Analysis must be cached to avoid unnecessary re-runs.

FR-3 Report Generation

ID	Requirement
FR-3.1	The report must be a single self-contained HTML folder.
FR-3.2	It must open in any modern browser.
FR-3.3	It must have 5 sections: Overview, Chakra Network, Techniques, Impact Sight, Files.
FR-3.4	All text must adapt to the user's profile.
FR-3.5	It must work offline after generation.
FR-3.6	Report data must load without a server, so <code>index.html</code> works when opened directly from disk.
FR-3.7	The Files section must offer "View Impact", and "Related Concepts" when Techniques data exists.

FR-4 Adaptive Explanations

ID	Requirement
FR-4.1	<code>byakugan explain <file></code> must produce an explanation in the terminal.
FR-4.2	Explanation depth must reflect the user's profile.
FR-4.3	In the report, changing the level must re-render explanations instantly.
FR-4.4	All three levels must be pre-computed during analysis.

FR-5 Impact Sight

ID	Requirement
FR-5.1	<code>byakugan impact <file></code> must produce an impact report in the terminal.
FR-5.2	It must list direct and indirect dependents.
FR-5.3	It must identify test coverage gaps.
FR-5.4	It must suggest a safe modification approach.
FR-5.5	It must flag risky patterns.

FR-6 Sharing

ID	Requirement
FR-6.1	<code>byakugan share</code> must deploy the report to Vercel.
FR-6.2	It must return a unique shareable URL.
FR-6.3	The URL must be copied to the clipboard.
FR-6.4	The deploy must complete within 60 seconds.

FR-7 Utility Commands

ID	Requirement
FR-7.1	<code>byakugan open</code> must open the report in the default browser.
FR-7.2	<code>byakugan status</code> must show profile, reports, and cache state.
FR-7.3	<code>byakugan clean</code> must remove cached files.
FR-7.4	<code>byakugan --help</code> must list all commands and examples.
FR-7.5	Every command must accept <code>--help</code> .

FR-8 Error Handling

ID	Requirement
FR-8.1	All errors must be explained in plain language.
FR-8.2	Errors must suggest a fix.
FR-8.3	The tool must never crash on malformed input.
FR-8.4	Ctrl+C must cleanly interrupt any command.

FR-9 Chat Widget (stretch, optional)

ID	Requirement
FR-9.1	A floating button on every page must open a chat panel.
FR-9.2	Messages must go to a serverless function that forwards them to Bob in Ask Mode (read-only).
FR-9.3	Context must be analysis data, the user's profile, and the current file. No raw source files.
FR-9.4	Answers must stream token by token.
FR-9.5	Answers must match the user's skill level.
FR-9.6	The chat must not modify code, run commands, or access files outside the analyzed codebase.
FR-9.7	The endpoint must be rate-limited.
FR-9.8	If the chat is disabled or unreachable, the button is hidden and the rest of the report is unaffected.

Part 6. Non-Functional Requirements

Non-functional requirements describe how well the tool must perform. They are about quality, not features.

Area	Requirements
NFR-1 Performance	Analysis under 5 minutes for a 200-file repo. <code>explain</code> under 30 seconds. <code>impact</code> under 60 seconds. Report opens in under 2 seconds (about 5 MB for a 200-file repo is acceptable).
NFR-2 Usability	A beginner can run all commands without reading docs. Every command has <code>-help</code> with examples. Errors are plain-language with a suggested fix. Progress is visible during long operations (spinners, step counters).
NFR-3 Reliability	No crash on malformed code. Graceful degradation if the Bob API fails. Partial results saved if analysis is interrupted. Re-running never corrupts the previous report. The report works fully without the optional chat.
NFR-4 Portability	macOS, Linux, and Windows. Only Node.js v20+ required. Works in bash, zsh, PowerShell, and cmd.
NFR-5 Security	Source code is sent only to the Bob API. No credentials in the report. <code>.gitignore</code> is respected when scanning. No source code in the report, only analysis. The optional chat uses analysis data only, keeps its Bob credential server-side, and is rate-limited.
NFR-6 Maintainability	Modular code (one file per concern). Every function commented. Prompts live in template files, not inline strings. Commit convention followed.
NFR-7 Visual quality	Tailwind CSS shipped locally, dark mode by default, diagrams that render without external dependencies, readable typography (Inter or similar).
NFR-8 Accessibility	Core content (the Overview) works without JavaScript. Color contrast meets WCAG AA. Keyboard navigation for all interactive elements.

Part 7. The Report

7.1 What the report is

The report is a **single-page application (SPA)** packaged as a self-contained folder: a mini-website that runs entirely in your browser. It needs no server to display. Open `index.html` and everything works.

Technically it is a **static site**: plain HTML, CSS, and JavaScript with no backend logic. What makes it feel like an app rather than a document is **client-side navigation**: JavaScript intercepts sidebar clicks and swaps the content without reloading the page.

```
.byakugan/output/report/
├── index.html      <- the entry point you open
├── assets/
│   ├── app.js      <- logic (navigation, interactivity)
│   ├── styles.css  <- visual design (Tailwind, local)
│   ├── mermaid.min.js <- diagram rendering library (local)
│   └── chat.js     <- STRETCH: chat widget (only if built)
└── data/
    ├── analysis.json <- the raw Bob output (all the content)
    └── analysis.js   <- the same data, loadable from disk
```

`analysis.json` is the **content**. The HTML and JavaScript files are the **shell** that displays it. Because they are separate, the report can be regenerated with new data without touching the code, and the code can be updated without re-running the analysis.

Design decisions in v1.1

Topic	Decision	Why
Data loading	Embed the data as <code>data/analysis.js</code> , a script that sets <code>window.BYAKUGAN_DATA</code> . Keep <code>analysis.json</code> as the raw copy.	Browsers block <code>fetch()</code> of local files, so a JSON-only design breaks when <code>index.html</code> is double-clicked.
Styling and diagrams	Ship Tailwind CSS and Mermaid as local files. No CDN links inside the report.	CDN links break offline use (FR-3.5). This replaces the CDN wording in v1.0.
Impact data	Build the dependency graph by parsing imports during analysis; the browser derives direct and indirect dependents from it. Bob supplies the explanation, blast radius, and plan for the most important files. (Recommended)	A full Bob impact report for every file will not fit the 5-minute analysis budget.
Overview without JavaScript	Pre-render the Overview text into <code>index.html</code> when the report is generated.	Meets NFR-8 for core content. Other sections need JavaScript.
Related Concepts button	Show it only when Techniques data exists.	Techniques is pitch-only, so the link would otherwise be dead.

7.2 The five sections

Each section is reached from the sidebar and answers one question.

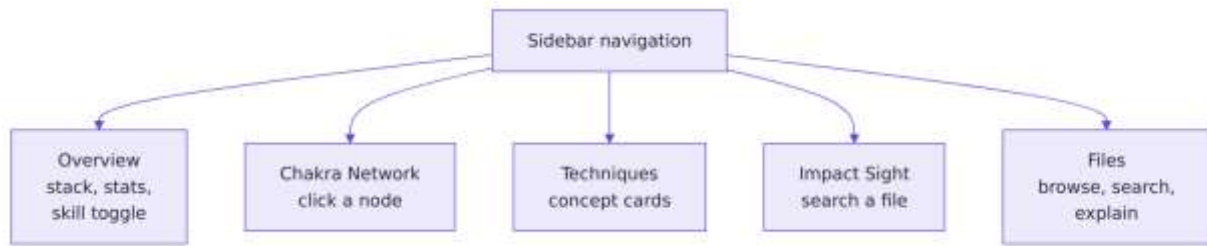


Figure 8. Report sections

Section 1: Overview. “What is this project?”

- Project name and a one-sentence description.
- A plain-language paragraph on what the project does.
- The tech stack (for example Node.js, Express, PostgreSQL).
- Quick stats: files, modules, concepts found, dependencies mapped.
- The **skill-level dropdown**, the key interactive element.

Item	Example Overview content
Project	payments-backend : a payments processing system for an e-commerce platform. Handles refunds, chargebacks, and transaction logging.
Tech stack	Node.js · Express · PostgreSQL · Redis
Stats	247 files · 12 modules · 5 concepts · 47 dependencies

Section 2: Chakra Network. “How is this project organized?”

A visual diagram where each box is a module and lines show which modules call which. Nodes are clickable if time allows. The diagram is drawn by **Mermaid.js**, which turns text produced during analysis into a graph, embedded directly in the page with no image files:

```
graph TD
  Auth --> User
  User --> DB
  Auth --> Log
```

Priority: stretch. If time is short, this section is cut or shown as a static diagram.

Section 3: Techniques. “What design ideas does this code use?”

A card for every design pattern or concept found (fields in section 4.4). “Show in code” jumps to the file in the Files section; “Show example” expands the snippet. **Priority: pitch only.**

Section 4: Impact Sight. “What breaks if I change this?”

A search box: type a file name and the report shows what it does, direct and indirect dependents, test coverage, blast radius, a suggested approach, and warnings (fields in section 4.5). **Priority: must build; this is the demo centerpiece.**

Section 5: Files. “Show me any file, explained at my level.”

- A searchable, filterable list of every file in the project.
- Click a file and its explanation appears, adapted to the selected skill level.
- “View Impact” jumps to Impact Sight for that file.

- “Related Concepts” jumps to Techniques (shown only when Techniques exists).

Sometimes you just want to look at one file without running a CLI command. This section gives you that. **Priority: P0, basic file browser with explanations.**

7.3 How the report works technically

The data flow

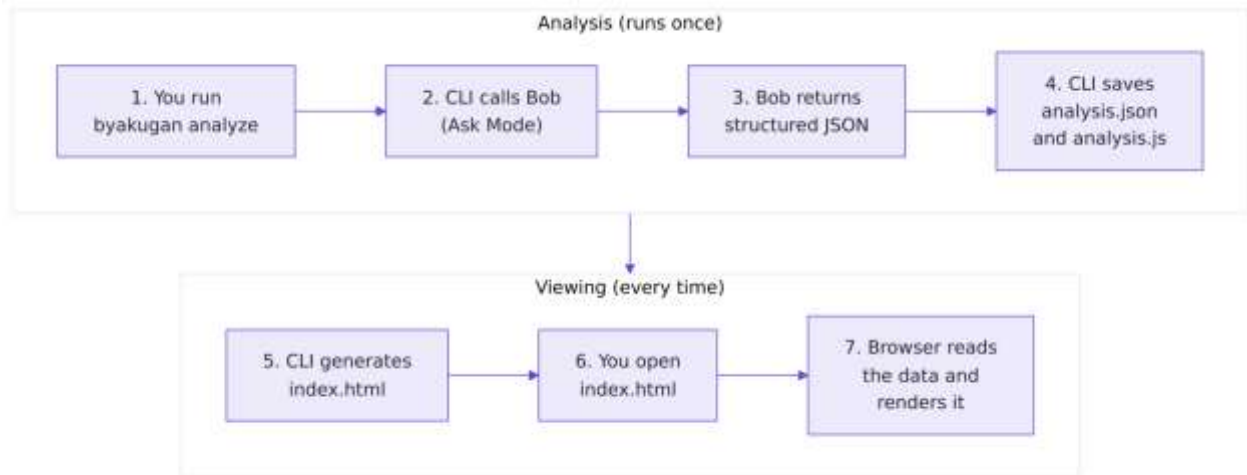


Figure 9. From `byakugan analyze` to an interactive report

Key insight: Bob runs **once per analysis**. All three skill levels are pre-computed, so the user toggles freely with instant response and no API calls. The one exception is the optional chat widget (section 4.6), which calls Bob live.

Client-side navigation

The report uses **hash-based routing**: the URL changes to `#overview`, `#impact`, `#files`, and JavaScript listens for the change and swaps the displayed content. This is why clicking a sidebar link does not reload the page, and why deep links to a section work.

The skill-level toggle

This is the interactive feature that makes the demo land. All three versions live in the data, and the toggle only swaps which one is displayed:

```
// All three versions live in the data
const explanation = {
  beginner: "Think of it like a librarian...",
  intermediate: "Repository pattern. Notice how...",
  senior: "Repository with a leaky abstraction..."
};

// Toggle just swaps which one is displayed
document.getElementById("explanation").innerText =
  explanation[selectedLevel];
```

Cost: the report is about 3x larger. For a 200-file repo that is roughly 5 MB instead of 1.5 MB, which is acceptable.

7.4 Interactivity

Everything is client-side JavaScript. No server is needed (except the optional chat).

Feature	User action	What happens
Sidebar navigation	Clicks a section	Content swaps instantly
File search	Types in a search box	File list filters live
Explain a file	Clicks a file name	Explanation expands inline
Change skill level	Uses a dropdown	All explanations re-render
View impact	Clicks “Impact” on a file	Impact report slides in
Copy code	Clicks a “copy” button	Code copied to clipboard
Deep-linking	Uses URL hash	Browser opens at that section
Dark/light toggle	Clicks sun/moon	Theme switches
Chat (stretch)	Clicks the floating button	Chat panel opens

7.5 Sharing the report

A static report is already shareable: zip it and email it, put it on a USB stick, upload it to Google Drive, or open it offline. But sharing a folder is clunky, so `byakugan share` deploys it to **Vercel** (free tier) and returns a unique URL that is copied to your clipboard.

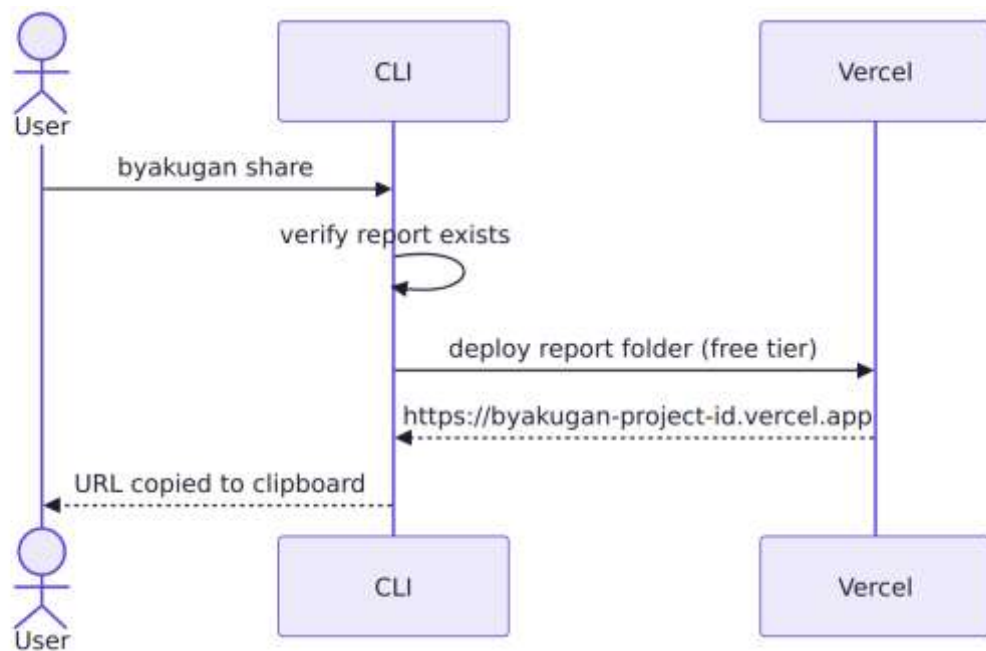


Figure 10. The byakugan share flow

The URL format is `https://byakugan-<project>-<id>.vercel.app`, for example `https://byakugan-payments-abc123.vercel.app`. The first time, the user is prompted to log into Vercel; after that, sharing is one command. Vercel is free, deploys from a folder in seconds, and is a service judges recognize. A shareable link turns Byakugan from “a tool I use” into “a tool my team uses”.

Before demo day: open a shared link in a private browser window and confirm it loads without a Vercel login. Check the project's Deployment Protection settings if it does not.

What the report is not

Question	Answer
Does it modify code?	No. Read-only.
Does it require internet?	No. It is a static HTML file. (Only the optional chat needs internet.)
Does it need a server to view?	No. Open the HTML file directly.
Does the share link expire?	Not by design. It stays live while the deployment exists.
Is the share link private?	No. Anyone with the link can view it. Do not share reports of private code publicly.
Can multiple people view at once?	Yes. It is just a web page.
Does it re-analyze on each view?	No. Everything is pre-computed.

Part 8. The CLI

8.1 The full command list

Group	Command	Purpose
Setup	<code>byakugan init</code>	First-time profile setup
Setup	<code>byakugan profile</code>	View or update profile
Analysis	<code>byakugan analyze <path></code>	Analyze a codebase
Analysis	<code>byakugan explain <file></code>	Explain one file
Analysis	<code>byakugan impact <file></code>	Show what breaks if you change this
Report	<code>byakugan open</code>	Open the report
Report	<code>byakugan share</code>	Get a shareable link
Utility	<code>byakugan status</code>	Show current state
Utility	<code>byakugan clean</code>	Remove generated files
Utility	<code>byakugan --help / --version</code>	Show all commands / show version

8.2 The end-to-end user journey

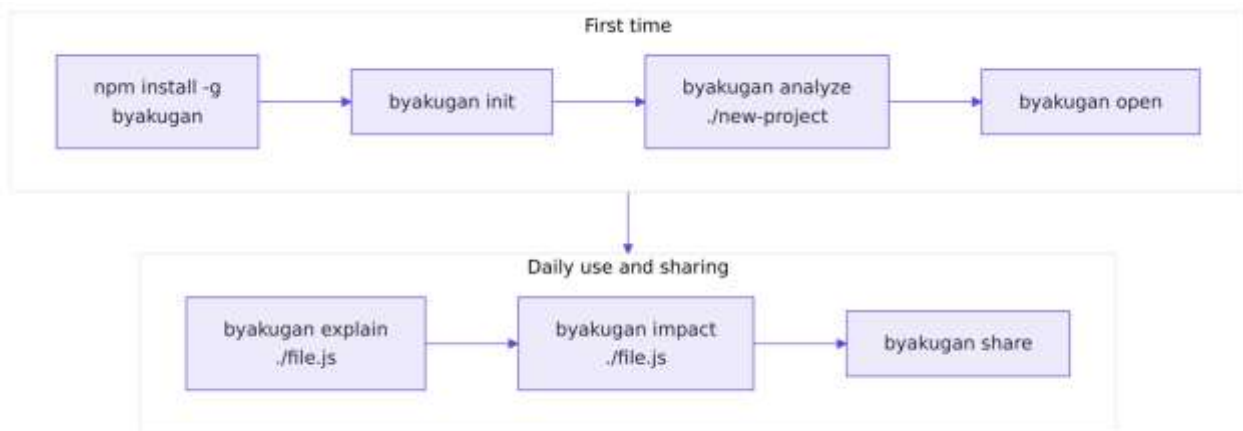


Figure 11. First-time and daily user journey

First time: install, profile yourself, analyze, open the report, and understand the project. **Daily use:** check one file with `explain`, verify safety with `impact` before a change. **Sharing:** run `byakugan share` and send the link to the team.

Part 9. Architecture

9.1 The big picture

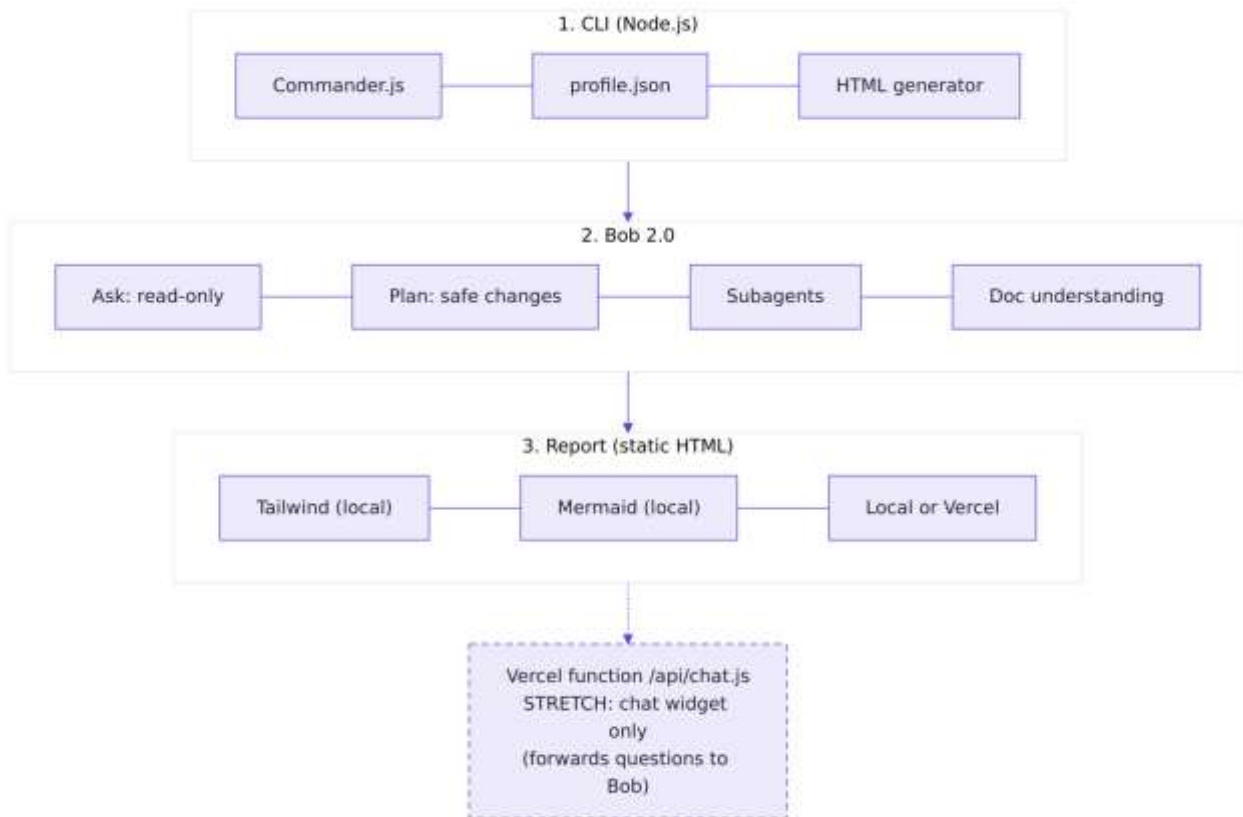


Figure 12. Layered architecture (dashed: optional chat function)

9.2 The analysis pipeline

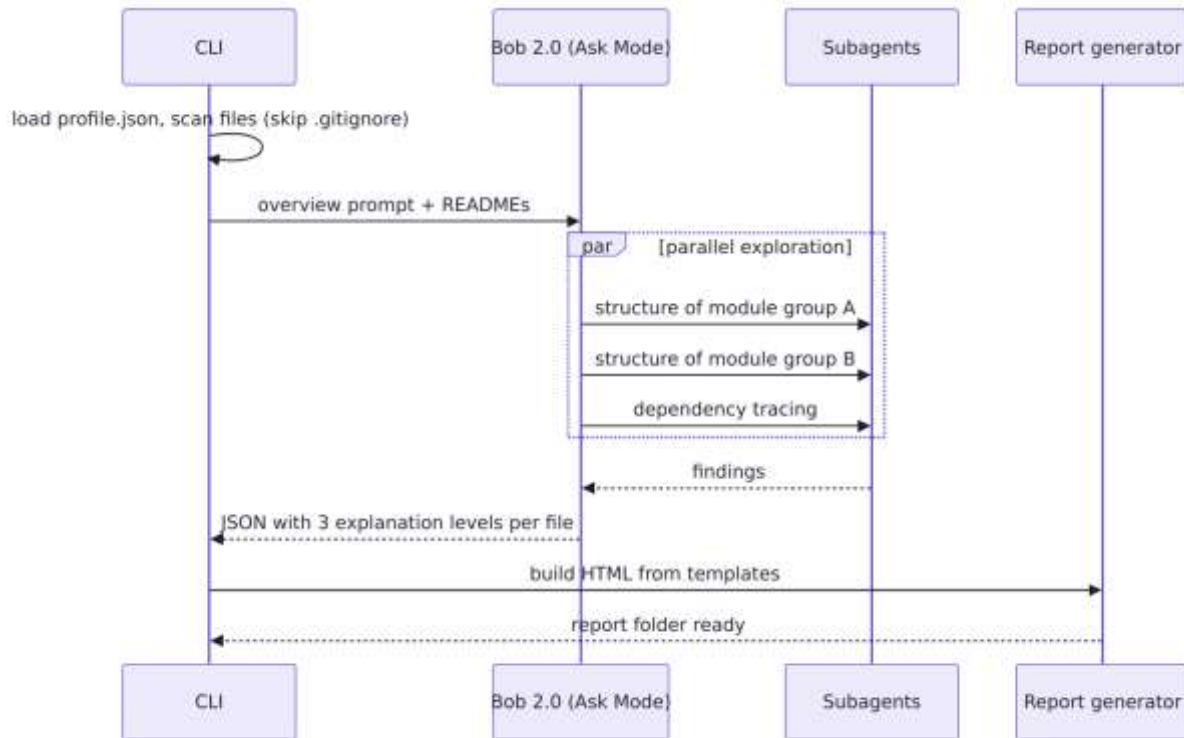


Figure 13. Sequence of *byakugan analyze* (proposed detail)

9.3 Data model (proposed)

`analysis.json` is the single source of truth for the report. Each file carries all three explanation levels so the report never needs an API call after generation.

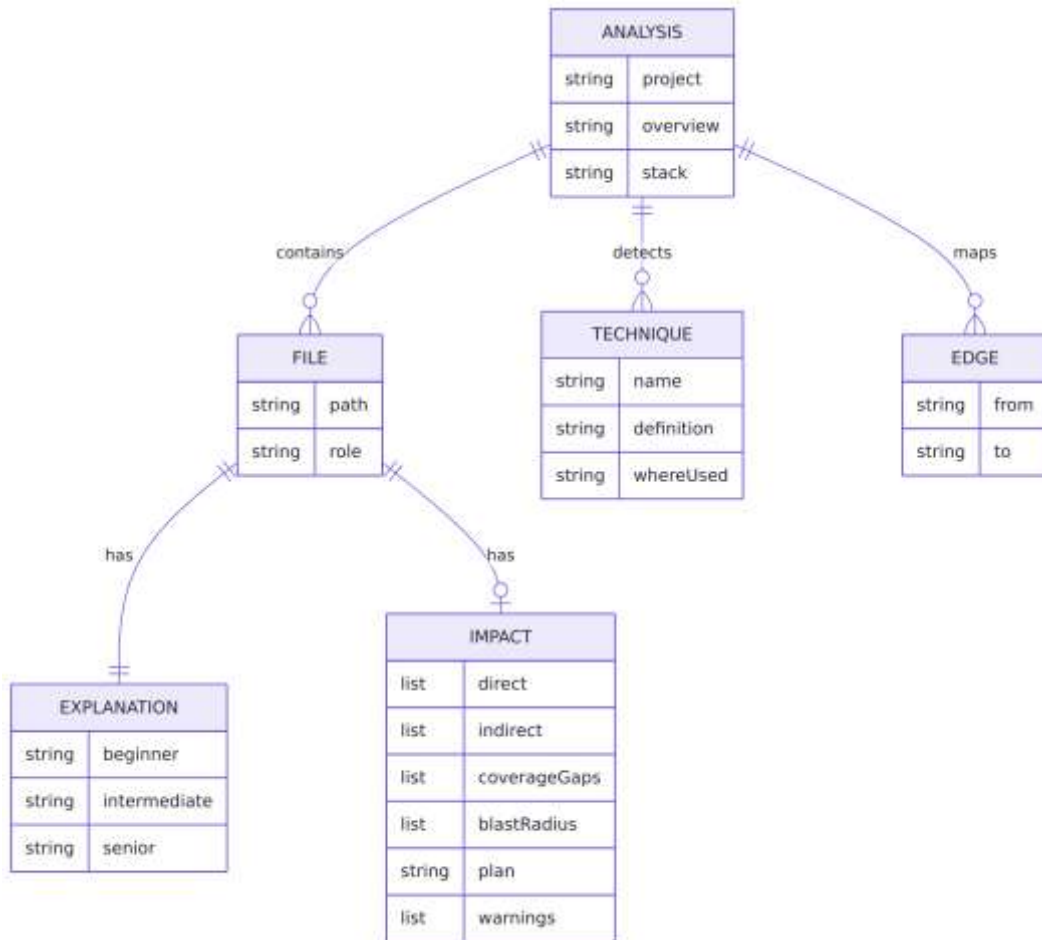


Figure 14. Proposed entities inside `analysis.json`

9.4 The project structure

```

byakugan/
├── bin/
│   └── byakugan.js           # CLI entry point
├── lib/
│   ├── profiler.js          # Skill profiling
│   ├── analyzer.js          # Bob API calls
│   ├── explainer.js         # Adaptive explanations
│   ├── impact.js            # Impact Sight
│   ├── reporter.js          # HTML generation
│   └── sharer.js            # Vercel deployment
├── api/
│   └── chat.js               # STRETCH: chat function (optional)
├── templates/
│   ├── prompts/             # Bob prompt templates
│   │   ├── overview.md
│   │   ├── explain.md
│   │   └── impact.md
│   └── report/
│       ├── index.html       # includes an empty chat-root placeholder
│       ├── app.js
│       ├── chat.js          # STRETCH: chat widget (optional)
│       └── styles.css
├── .byakugan/               # Local data (gitignored)
│   ├── profile.json
│   └── output/
├── package.json
└── README.md

```

9.5 How Bob 2.0 is used

Judges want to see that we used Bob’s real capabilities, not just called an LLM.

Bob feature	Where we use it	Why
Ask Mode	Overview, Explain, Impact	Read-only; safe exploration
Plan Mode	Impact Sight “suggested approach”	Structured proposal before action
Agent Mode	HTML report generation	Generation task
Subagents	Structure analysis, impact tracing	Parallel exploration, clean context
Document understanding	Overview	Ingest READMEs and docs
Parallel tool calls	Structure analysis	Faster cross-file reads
270k context window	Large repos	Full codebase in one context
Ask Mode, streaming	Chat widget (stretch)	Read-only answers in context

Part 10. Technology

10.1 The tech stack

Layer	Tool	Why we chose it
Language	Node.js v20+	Natural CLI fit; easy Bob integration
CLI framework	Commander.js	Minimal, well documented
Bob integration	Bob CLI / pi-bob	Handles SSO and token refresh
Report UI	HTML + Tailwind CSS + Mermaid.js (shipped locally)	Works offline; no CDN dependency
Deployment	Vercel (report) + GitHub (CLI)	Free tier; zero config
Chat function (stretch)	Vercel serverless function	Same platform as sharing; only if the chat is built
Version control	GitHub	Required for Vercel

10.2 Environment setup

1. **Install Node.js v20+.** Download the LTS version from nodejs.org and install with defaults.
2. **Install IBM Bob 2.0.** Follow IBM's installation guide and verify SSO works.
3. **Verify Bob API.** Send a test message and confirm you get a response.
4. **Clone the repo.** `git clone <repo-url>`, `cd byakugan`, `npm install`.
5. **Test the CLI.** `node bin/byakugan.js --help`.

Part 11. The 48-Hour Plan

11.1 Phase 0 (before the hackathon)

Do this together before the clock starts (2–3 hours):

- Install Node.js v20+ on both machines.
- Install IBM Bob 2.0 on both machines.
- Verify Bob API access on both machines.
- Agree on the demo repo.
- Create the GitHub repo and add both as collaborators.
- Both read this document.
- Agree on the tagline and visual direction.
- Feasibility check (15 minutes): can Bob be called from a server with a non-interactive credential? This decides whether the chat widget is possible at all.

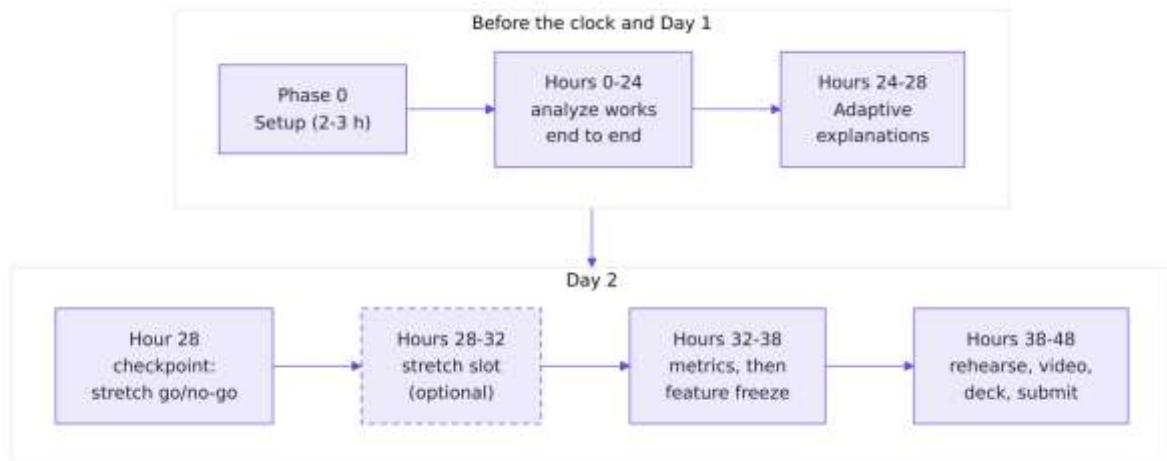


Figure 15. Timeline overview

11.2 Day 1 (hours 0–24)

Hours	Lead	Teammate
0–2	Project setup, install dependencies	Read Bob docs, understand modes
2–6	Bob integration layer	Write and test 3 core prompts
6–10	Impact Sight feature	Design report layout (paper sketch)
10–14	Wire CLI commands	Build HTML template
14–20	Skill Profiling	Fill report with real data
20–24	Run on demo repo, fix bugs	Run on demo repo, fix bugs

Goal by hour 24: `byakugan analyze ./demo-repo` works end to end.

11.3 Day 2 (hours 24–48)

Hours	Lead	Teammate
24–28	Adaptive explanations	Polish visuals, styles
28–32	Stretch slot after the go/no-go check: Chakra Network or chat backend (see 11.4). Otherwise polish.	Write demo script (plus chat panel UI if chat is attempted)
32–38	Capture metrics	Capture metrics
38–42	Rehearse demo	Rehearse demo
42–45	Record backup video	Prepare pitch deck
45–48	Final testing, submit	Final testing, submit

Hard rule: no new features after hour 38. Polish only.

11.4 Stretch goals and the hour-28 checkpoint

Two features compete for the stretch slot: the Chakra Network and the chat widget. To keep the core safe, the decision is made at a fixed checkpoint with a fixed time-box.

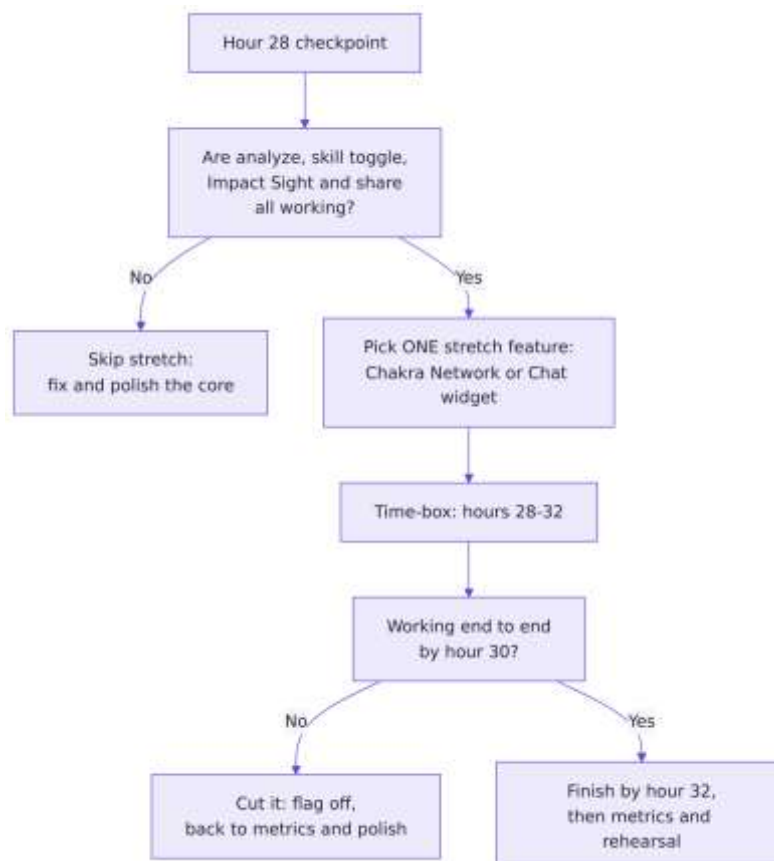


Figure 16. Stretch go/no-go decision

Aspect	Chakra Network	Chat widget
Effort	Low: diagram text is already produced during analysis	Medium to high: widget, serverless function, secret, rate limit
Risk	Low	Medium to high: depends on server access to Bob
Demo impact (team judgment)	Medium: shows the project's shape	High in a live demo, but more can go wrong
Depends on	Nothing extra	Phase 0 feasibility check passing

- **Pick one** stretch feature at the checkpoint. If the team is ahead of schedule at hour 24, attempt both, with the chat second.
- **The Lead decides.** The checklist is: analyze works, the skill toggle works, Impact Sight works with real data, and share deploys.
- **Kill rule:** if the feature is not working end to end by hour 30 (for the chat, one full question-and-answer round trip), turn its flag off and go back to metrics and polish.
- **Nothing is wasted:** stretch code is isolated, so cutting it never breaks the core.

Part 12. Team Roles

Role	Person	Owns
Lead	You (1 yr enterprise)	Bob integration, CLI, architecture
Prompt & Report	Teammate (beginner)	Prompts, HTML, visuals
Demo	Both	Script, rehearsal

If the chat widget is attempted: the Lead builds `api/chat.js`; the teammate builds the panel UI in `chat.js`. One person still owns one file at a time.

Communication rules

Small teams fail from misalignment, not lack of skill.

1. **Sync every 4 hours**, 15 minutes max. Three questions: what did you finish, what is blocking you, what is next?
2. **Ask for help within 30 minutes** of being stuck. No silent struggling.
3. **Commit early, commit often.** Even broken code.
4. **One person owns one file at a time.** No simultaneous edits.
5. **If you disagree, the Lead decides** and you move on. Debates cost more than bad decisions.

Part 13. Deliverables and Success Metrics

13.1 What we submit

- GitHub repo with full source code.
- Working CLI: `byakugan analyze ./repo`.
- HTML report generator.
- Live demo on Vercel.
- Chat widget (only if built; otherwise shown in the deck as roadmap).
- Backup demo video (3–5 minutes).
- Pitch deck (5–7 slides).
- Metrics report (real numbers from a real repo).

13.2 Success metrics

Metric	How we measure	Target
Analysis time	Stopwatch	Under 5 minutes
Time to understand a feature	Stopwatch	Under 30 minutes
Concepts identified	Count from report	5–15
Dependents found	Count from Impact Sight	3–20
PR rounds reduced	Compare first PR with vs without	3 → 1

Part 14. Glossary

Term	What it means
CLI	Command Line Interface. The window where you type commands.
Repo	Repository. A project folder tracked by Git.
npm	Node Package Manager. Installs JavaScript tools.
Bob 2.0	IBM's AI development agent. Our "brain."
Ask Mode	Bob mode that only reads, never changes.
Plan Mode	Bob mode that produces a plan for review.
Agent Mode	Bob mode that can take action.
Subagent	A mini-Bob that explores one thing and reports back.
Hook	An automatic action triggered by an event.
Mermaid.js	A tool that turns text into diagrams.
Tailwind CSS	A styling tool for web pages.
Vercel	A free service that hosts websites.
CDN	Content Delivery Network. Loads tools from the internet.
JSON	A structured data format. Looks like {"name": "value"}.
Profile	Your saved skill level and preferences.
Analysis	The process of reading and understanding a codebase.
Report	The HTML output you read in your browser.
Impact Sight	The feature that shows what breaks if you change a file.
SPA	Single-page application. A web app that swaps content in place instead of loading new pages.
Static site	A site made only of HTML, CSS, and JavaScript files; no backend needed.
Hash routing	Navigation driven by the part of the URL after #, such as #impact.
Serverless function	A small piece of server code that runs on demand, for example on Vercel.
Streaming	Sending an answer piece by piece as it is generated.
Stretch goal	A feature attempted only if the core is finished and stable.

Part 15. One-Page Summary

What we are building: a command-line tool that turns any unfamiliar codebase into a personalized, interactive report.

The problem we solve: developers waste 1–2 weeks understanding code they inherited.

How we solve it: Byakugan profiles the reader, adapts explanations, teaches concepts in context, and shows what breaks before you change anything.

The five features:

1. Skill Profiling
2. Adaptive Explanations
3. Chakra Network (dependency map)
4. Technique Cards (concept learning)
5. Impact Sight (safe modification)

Optional sixth (stretch): a chat widget inside the report for follow-up questions.

What we build for the hackathon: features 1, 2, and 5 in depth; feature 4 as a pitch mockup only. **Stretch (one, decided at hour 28):** feature 3 (Chakra Network) or the optional chat widget.

The tech: Node.js CLI + Bob 2.0 + static HTML report, deployed via Vercel for sharing.

How we win: a universal problem, real Bob 2.0 usage (Ask/Plan/Agent modes, subagents), a demo in 5 minutes, and real metrics: 2 weeks → 30 minutes.